

8-bit Processor Project Report

Gabriel Huynh

November 30, 2025

Contents

1	Introduction	3
2	Components: Register, 4:16 Decoder, FSM	3
2.1	Register	3
2.2	4:16 Decoder	4
2.3	FSM	5
3	ALU Problem Set 1	6
4	ALU Problem Set 2	7
5	ALU Problem Set 3	8
6	Conclusion	8

1 Introduction

The Arithmetic Logic Unit (ALU), the core component in this lab, performs various arithmetic and boolean operations on two 8-bit inputs based on an operational code input. The two inputs are inputted to a register, which acts as a storage for the inputs, then fed into the ALU to ensure stable inputs. The Finite State Machine (FSM) is the component that creates the operational code needed for the ALU. It cycles through 9 different states in sequential order and outputs a 4-bit student number digit and a 4-bit operational code. The operational code is then fed into a 4:16 decoder so that the ALU can read exactly which bit is high to execute that specific operation.

2 Components: Register, 4:16 Decoder, FSM

2.1 Register

The register is a digital component that is constructed from multiple (in this case, 8) D flip-flops that are connected sequentially to hold the value of the input, with each D flip-flop corresponding to 1 bit. The register includes an asynchronous reset input *Resetn* that, when LOW, will force the stored value to 0. When *Resetn* is HIGH, the register will continually update its stored 8-bit value from the input every rising edge of the clock signal.

Resetn	Clock	Event	Q(next)
0	0	reset	00000000
0	1	reset	00000000
1	↑	rising edge	A
1	0 steady	no edge	Q
1	1 steady	no edge	Q
1	↓	falling edge	Q

Table 1: Truth table for Register

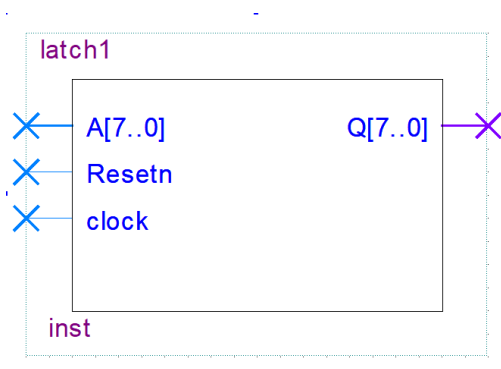


Figure 1: Block Diagram for Register

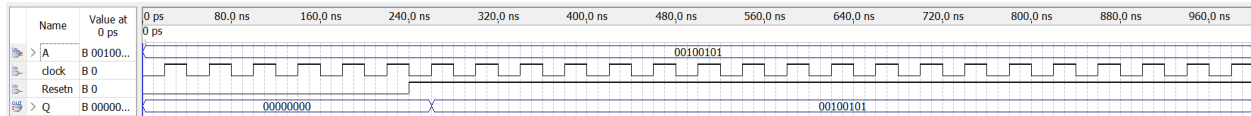


Figure 2: Waveform for Register

2.2 4:16 Decoder

The 4:16 decoder is a digital component that takes a 4-bit input and converts it into 2^4 (16) unique outputs, where only 1 bit of the 16-bit output can be HIGH at any given time. This component creates a more readable set of microcode instructions that the ALU can use to select the desired function.

Note: for the function of this lab, the 4:16 decoder will never take an input higher than 1000_2 .

Input	Enable	Output
X	0	0000000000000000
0000	1	0000000000000001
0001	1	0000000000000010
0010	1	0000000000000100
0011	1	0000000000001000
0100	1	0000000000010000
0101	1	0000000000100000
0110	1	0000000001000000
0111	1	0000000010000000
1000	1	0000000100000000

Table 2: Truth table for 4:16 Decoder

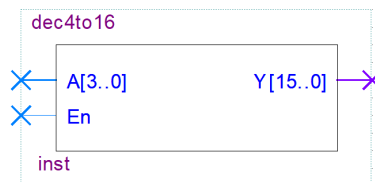


Figure 3: Block Diagram for 4:16 Decoder

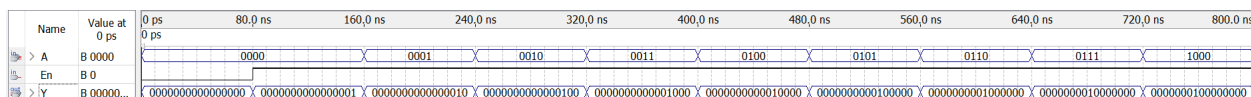


Figure 4: Waveform for 4:16 Decoder

2.3 FSM

The Finite State Machine is a digital component that cycles through a finite number of states (in this case, 9) in a controlled and predictable way. The FSM determines the next state based off of the current state and whether *data_in* is HIGH (next) or LOW (stay). Because this FSM is a Moore machine, the output logic only depends on the current state and not the input.

Present State	std_id	current_state	next state (<i>data_in</i> = 0)	next state (<i>data_in</i> = 1)
s0	5	0	s0	s1
s1	0	1	s1	s2
s2	1	2	s2	s3
s3	3	3	s3	s4
s4	0	4	s4	s5
s5	6	5	s5	s6
s6	2	6	s6	s7
s7	5	7	s7	s8
s8	7	8	s8	s0

Table 3: Truth table for FSM

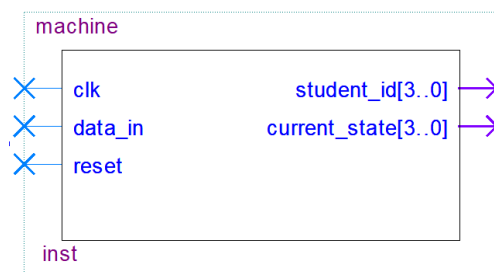


Figure 5: Block Diagram for FSM

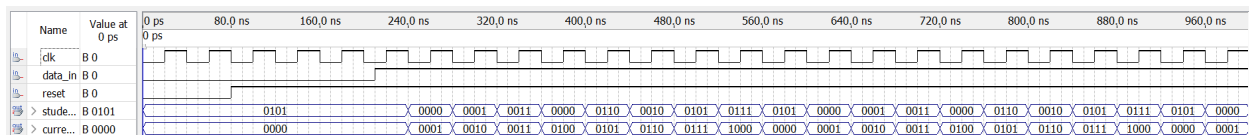


Figure 6: Waveform for FSM

3 ALU Problem Set 1

The ALU for problem set 1 simply executes a given function for 2 8-bit inputs (A and B) depending on the operational code given to it from the translated-FSM current_state output.

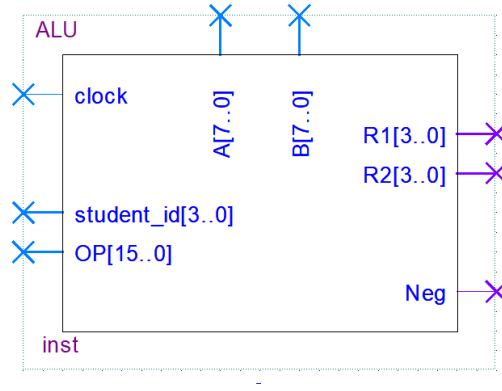


Figure 7: Block Diagram for ALU Problem Set 1

As stated above, the two inputs that the ALU operates on are A and B, which are 8-bit numbers from the two registers. OP is the 16-bit operational code from the FSM. R1 and R2 are the two components of the result, split up to fit on 2 seven-segment displays. Neg becomes HIGH when the value of the result is negative. Clock is just the clock signal for all components. Student_id is not used in this problem set.

Function #	Microcode	Boolean Operation / Function
1	0000000000000001	$\text{sum}(A, B)$
2	0000000000000010	$\text{diff}(A, B)$
3	0000000000000100	$\overline{A}$
4	0000000000001000	$\overline{A \cdot B}$
5	0000000000010000	$\overline{A + B}$
6	0000000000100000	$A \cdot B$
7	0000000001000000	$A \oplus B$
8	0000000010000000	$A + B$
9	0000000100000000	$\overline{A \oplus B}$

Figure 8: Microcode Instructions for ALU Problem Set 1

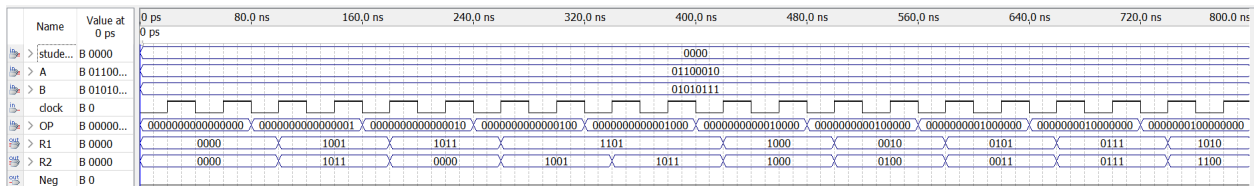


Figure 9: Waveform for ALU Problem Set 1

4 ALU Problem Set 2

Similar to Problem Set 1, the ALU for problem set 2 simply executes a given function for 2 8-bit inputs (A and B) depending on the operational code given to it from the translated-FSM current_state output.

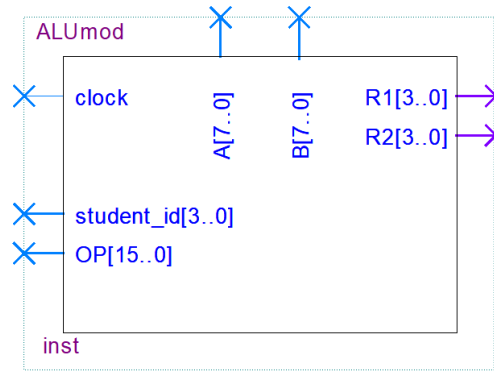


Figure 10: Block Diagram for ALU Problem Set 2

The Neg output is no longer needed as no microcode operations will result in negative numbers. See Problem Set 1 for the purpose of all other input/outputs.

Function #	Microcode	Operation
1	000000000001	A+2
2	000000000010	Shift B right 2 bits, input bit '0'
3	000000000100	Shift A right 4 bits, input bit '1'
4	000000001000	Min(A,B)
5	000000010000	Rotate A right 2 bits
6	000000100000	Invert bit-significance order of B
7	000001000000	A XOR B
8	000010000000	(A+B) - 4
9	000100000000	All bits high output

Table 4: Microcode for ALU Problem Set 2

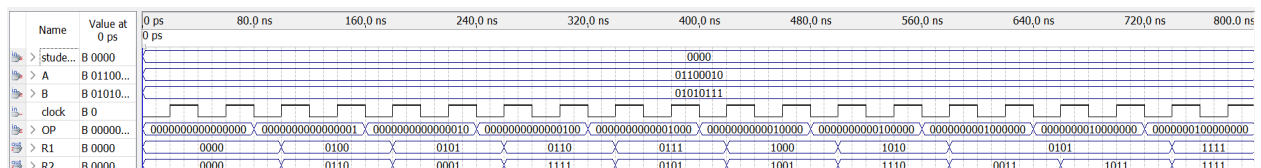


Figure 11: Waveform for ALU Problem Set 2

5 ALU Problem Set 3

The ALU for problem set 3 does not have multiple functions depending on the operational code anymore. Instead, it takes in the 4-bit current student number digit from the FSM, determines whether it is odd, and outputs "0001" if so and "0000" if not.

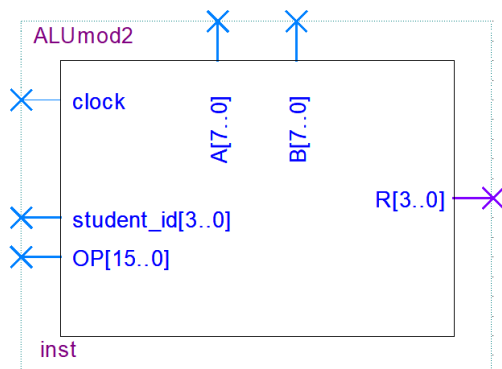


Figure 12: Block Diagram for ALU Problem Set 3

The student_id takes the student ID digit from the FSM as the input. The new R is the single 4-bit output to the seven segment display to show a 'y' or 'n'. The clock, AB inputs, and operational code remain the same (see Problem Set 1 or 2).

Function #	Microcode	Operation
1	X	odd(A)

Table 5: Microcode for ALU Problem Set 3

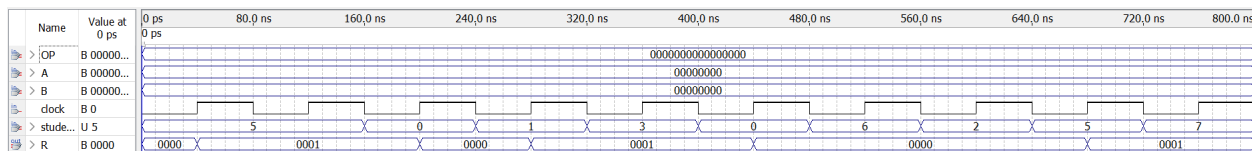


Figure 13: Waveform for ALU Problem Set 3

6 Conclusion

The general-purpose processor created in this lab utilizes registers, a control unit consisting of a Finite State Machine and a 4:16 decoder, an ALU core, and seven-segment displays. The processor performs various calculations using the microcode from the control unit, all operating around a centralized clock signal.